

Assignment-2

1. Write a C program to perform realization of Singly linked list.

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *next;
};

struct Node *head = NULL;

/* Function to allocate memory for a node */
struct Node* getNode() {
    return (struct Node*)malloc(sizeof(struct Node));
}

void insertAtBeginning(int value) {
    struct Node *newNode = getNode();
    newNode->data = value;
    newNode->next = head;
    head = newNode;
}

void insertAtEnd(int value) {
    struct Node *newNode = getNode();
    struct Node *temp = head;

    newNode->data = value;
    newNode->next = NULL;

    if (head == NULL) {
        head = newNode;
        return;
    }

    while (temp->next != NULL)
        temp = temp->next;

    temp->next = newNode;
}

void deleteFromBeginning() {
    if (head == NULL) {
        printf("List is empty\n");
        return;
    }
}
```

```

    struct Node *temp = head;
    head = head->next;
    free(temp);
}

void deleteFromEnd() {
    if (head == NULL) {
        printf("List is empty\n");
        return;
    }

    if (head->next == NULL) {
        free(head);
        head = NULL;
        return;
    }

    struct Node *temp = head;
    while (temp->next->next != NULL)
        temp = temp->next;

    free(temp->next);
    temp->next = NULL;
}

void display() {
    struct Node *temp = head;

    if (head == NULL) {
        printf("List is empty\n");
        return;
    }

    while (temp != NULL) {
        printf("%d ", temp->data);
        temp = temp->next;
    }
    printf("\n");
}

int main() {
    int choice, value;

    while (1) {
        printf("\n1.Insert at Beginning");
        printf("\n2.Insert at End");
        printf("\n3.Delete from Beginning");
    }
}

```

```
printf("\n4.Delete from End");
printf("\n5.Display");
printf("\n6.Exit\n");

scanf("%d", &choice);

switch (choice) {
    case 1:
        scanf("%d", &value);
        insertAtBeginning(value);
        break;

    case 2:
        scanf("%d", &value);
        insertAtEnd(value);
        break;

    case 3:
        deleteFromBeginning();
        break;

    case 4:
        deleteFromEnd();
        break;

    case 5:
        display();
        break;

    case 6:
        printf("Program Terminated\n");
        exit(0);
}
}
return 0;
}
```

Output:

- 1.Insert at Beginning
- 2.Insert at End
- 3.Delete from Beginning
- 4.Delete from End
- 5.Display
- 6.Exit

1
10
2
20
2
30

5
10 20 30

4

5
10 20

3

5
20

6
Program Terminated

2. Write a C program to perform realization of Doubly linked list.

```
#include <stdio.h>
#include <stdlib.h>
```

```
struct Node {
    int data;
    struct Node *prev;
    struct Node *next;
};
```

```
typedef struct Node* NodePtr;
```

```
NodePtr head = NULL;
```

```
/* getNode with direct return (no if, no variable) */
NodePtr getNode() {
    return (NodePtr)malloc(sizeof(struct Node));
}
```

```
void insertAtBeginning(int value) {
```

```

NodePtr newNode = getNode();

newNode->data = value;
newNode->prev = NULL;
newNode->next = head;

if (head != NULL)
    head->prev = newNode;

head = newNode;
}

```

```

void insertAtEnd(int value) {
    NodePtr newNode = getNode();
    NodePtr temp = head;

    newNode->data = value;
    newNode->next = NULL;

    if (head == NULL) {
        newNode->prev = NULL;
        head = newNode;
        return;
    }

    while (temp->next != NULL)
        temp = temp->next;

    temp->next = newNode;
    newNode->prev = temp;
}

```

```

void deleteFromBeginning() {
    if (head == NULL) {
        printf("List is empty\n");
        return;
    }
}

```

```

NodePtr temp = head;
head = head->next;

if (head != NULL)
    head->prev = NULL;

free(temp);
}

```

```

void deleteFromEnd() {

```

```

if (head == NULL) {
    printf("List is empty\n");
    return;
}

if (head->next == NULL) {
    free(head);
    head = NULL;
    return;
}

NodePtr temp = head;
while (temp->next != NULL)
    temp = temp->next;

temp->prev->next = NULL;
free(temp);
}

void display() {
    NodePtr temp = head;

    if (temp == NULL) {
        printf("List is empty\n");
        return;
    }

    while (temp != NULL) {
        printf("%d ", temp->data);
        temp = temp->next;
    }
    printf("\n");
}

int main() {
    int choice, value;

    while (1) {
        printf("\n1.Insert at Beginning");
        printf("\n2.Insert at End");
        printf("\n3.Delete from Beginning");
        printf("\n4.Delete from End");
        printf("\n5.Display");
        printf("\n6.Exit\n");

        scanf("%d", &choice);

        switch (choice) {

```

```

        case 1:
            scanf("%d", &value);
            insertAtBeginning(value);
            break;

        case 2:
            scanf("%d", &value);
            insertAtEnd(value);
            break;

        case 3:
            deleteFromBeginning();
            break;

        case 4:
            deleteFromEnd();
            break;

        case 5:
            display();
            break;

        case 6:
            printf("Program Terminated\n");
            exit(0);
    }
}
return 0;
}

```

Output:

```

1.Insert at Beginning
2.Insert at End
3.Delete from Beginning
4.Delete from End
5.Display
6.Exit
1
10
2
20
2
30
5
10 20 30

```

```
4
5
10 20

3
5
20

6
Program Terminated
```

3. Write a C program to perform realization of Circular Singly linked list.

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *next;
};

typedef struct Node* NodePtr;

NodePtr tail = NULL;

/* getNode with direct return, no if, no extra variable */
NodePtr getNode() {
    return (NodePtr)malloc(sizeof(struct Node));
}

void insertAtBeginning(int value) {
    NodePtr newNode = getNode();
    newNode->data = value;

    if (tail == NULL) {
        tail = newNode;
        tail->next = tail;
    } else {
        newNode->next = tail->next;
        tail->next = newNode;
    }
}

void insertAtEnd(int value) {
    NodePtr newNode = getNode();
```

```

newNode->data = value;

if (tail == NULL) {
    tail = newNode;
    tail->next = tail;
} else {
    newNode->next = tail->next;
    tail->next = newNode;
    tail = newNode;
}
}

```

```

void deleteFromBeginning() {
    if (tail == NULL)
        return;

    NodePtr head = tail->next;

    if (tail == head) {
        free(tail);
        tail = NULL;
    } else {
        tail->next = head->next;
        free(head);
    }
}

```

```

void deleteFromEnd() {
    if (tail == NULL)
        return;

    NodePtr head = tail->next;

    if (head == tail) {
        free(tail);
        tail = NULL;
    } else {
        NodePtr temp = head;
        while (temp->next != tail)
            temp = temp->next;

        temp->next = head;
        free(tail);
        tail = temp;
    }
}

```

```

void display() {

```

```

if (tail == NULL) {
    printf("List is empty\n");
    return;
}

NodePtr temp = tail->next;
do {
    printf("%d ", temp->data);
    temp = temp->next;
} while (temp != tail->next);
printf("\n");
}

int main() {
    int choice, value;

    while (1) {
        printf("\n1.Insert at Beginning");
        printf("\n2.Insert at End");
        printf("\n3.Delete from Beginning");
        printf("\n4.Delete from End");
        printf("\n5.Display");
        printf("\n6.Exit\n");

        scanf("%d", &choice);

        switch (choice) {
            case 1:
                scanf("%d", &value);
                insertAtBeginning(value);
                break;

            case 2:
                scanf("%d", &value);
                insertAtEnd(value);
                break;

            case 3:
                deleteFromBeginning();
                break;

            case 4:
                deleteFromEnd();
                break;

            case 5:
                display();
                break;

```

```

        case 6:
            printf("Program Terminated\n");
            exit(0);
        }
    }
    return 0;
}

```

Output:

```

1.Insert at Beginning
2.Insert at End
3.Delete from Beginning
4.Delete from End
5.Display
6.Exit
1
10
2
20
2
30

5
10 20 30

4

5
10 20

3

5
20

6
Program Terminated

```

4. Write a C program to perform realization of Circular Doubly linked list.

```

#include <stdio.h>
#include <stdlib.h>

struct Node {

```

```

    int data;
    struct Node *prev;
    struct Node *next;
};

typedef struct Node* NodePtr;

NodePtr head = NULL;

/* getNode with direct return */
NodePtr getNode() {
    return (NodePtr)malloc(sizeof(struct Node));
}

void insertAtBeginning(int value) {
    NodePtr newNode = getNode();
    newNode->data = value;

    if (head == NULL) {
        newNode->next = newNode;
        newNode->prev = newNode;
        head = newNode;
    } else {
        NodePtr last = head->prev;

        newNode->next = head;
        newNode->prev = last;

        head->prev = newNode;
        last->next = newNode;

        head = newNode;
    }
}

void insertAtEnd(int value) {
    NodePtr newNode = getNode();
    newNode->data = value;

    if (head == NULL) {
        newNode->next = newNode;
        newNode->prev = newNode;
        head = newNode;
    } else {
        NodePtr last = head->prev;

        newNode->next = head;
        newNode->prev = last;
    }
}

```

```

        last->next = newNode;
        head->prev = newNode;
    }
}

void deleteFromBeginning() {
    if (head == NULL)
        return;

    if (head->next == head) {
        free(head);
        head = NULL;
    } else {
        NodePtr last = head->prev;
        NodePtr temp = head;

        head = head->next;
        head->prev = last;
        last->next = head;

        free(temp);
    }
}

void deleteFromEnd() {
    if (head == NULL)
        return;

    if (head->next == head) {
        free(head);
        head = NULL;
    } else {
        NodePtr last = head->prev;
        NodePtr beforeLast = last->prev;

        beforeLast->next = head;
        head->prev = beforeLast;

        free(last);
    }
}

void display() {
    if (head == NULL) {
        printf("List is empty\n");
        return;
    }
}

```

```

NodePtr temp = head;
do {
    printf("%d ", temp->data);
    temp = temp->next;
} while (temp != head);

printf("\n");
}

int main() {
    int choice, value;

    while (1) {
        printf("\n1.Insert at Beginning");
        printf("\n2.Insert at End");
        printf("\n3.Delete from Beginning");
        printf("\n4.Delete from End");
        printf("\n5.Display");
        printf("\n6.Exit\n");

        scanf("%d", &choice);

        switch (choice) {
            case 1:
                scanf("%d", &value);
                insertAtBeginning(value);
                break;

            case 2:
                scanf("%d", &value);
                insertAtEnd(value);
                break;

            case 3:
                deleteFromBeginning();
                break;

            case 4:
                deleteFromEnd();
                break;

            case 5:
                display();
                break;

            case 6:
                printf("Program Terminated\n");

```

```
        exit(0);
    }
}
return 0;
}
```

Output:

```
1.Insert at Beginning
2.Insert at End
3.Delete from Beginning
4.Delete from End
5.Display
6.Exit
1
10
2
20
2
30

5
10 20 30

4

5
10 20

3

5
20

6
Program Terminated
```

Assignment-3

1. Write a C program to perform realization of integer static Stack.

```
#include <stdio.h>
#include <stdlib.h>

#define STACKSIZE 5 /* change size as needed */

struct stack {
    int top;
    int item[STACKSIZE];
} s;

/* function prototypes */
void push(int x);
void pop();
void display();
int isEmpty();
int isFull();

int main() {
    int choice, x;
    s.top = -1; /* initialize stack */

    while (1) {
        printf("\nMENU\n");
        printf("1. PUSH\n");
        printf("2. POP\n");
        printf("3. DISPLAY\n");
        printf("4. EXIT\n");
        printf("Enter your choice: ");
        if (scanf("%d", &choice) != 1) { /* basic input guard */
            printf("Invalid input.\n");
            exit(1);
        }

        switch (choice) {
            case 1:
                printf("Enter the number to be pushed: ");
                scanf("%d", &x);
                push(x);
                break;
            case 2:
                pop();
                break;
            case 3:
                display();
                break;
            case 4:
```

```

        printf("Program Terminated\n");
        exit(0);
    default:
        printf("Invalid choice\n");
    }
}
return 0;
}

void push(int x) {
    if (isFull()) {
        printf("Stack overflow\n");
    } else {
        s.item[++s.top] = x;
        printf("%d pushed into stack\n", x);
    }
}

void pop() {
    if (isEmpty()) {
        printf("Stack underflow\n");
    } else {
        printf("Deleted element is %d\n", s.item[s.top--]);
    }
}

void display() {
    int i;
    if (isEmpty()) {
        printf("Stack is empty\n");
        return;
    }
    printf("Stack elements (top to bottom):\n");
    for (i = s.top; i >= 0; --i)
        printf("%d ", s.item[i]);
    printf("\n");
}

int isEmpty() {
    return (s.top == -1);
}

int isFull() {
    return (s.top == STACKSIZE - 1);
}

```

Output:

MENU

1. PUSH
2. POP
3. DISPLAY
4. EXIT

Enter your choice: 1

Enter the number to be pushed: 10

10 pushed into stack

MENU

1. PUSH
2. POP
3. DISPLAY
4. EXIT

Enter your choice: 1

Enter the number to be pushed: 20

20 pushed into stack

MENU

1. PUSH
2. POP
3. DISPLAY
4. EXIT

Enter your choice: 1

Enter the number to be pushed: 30

30 pushed into stack

MENU

1. PUSH
2. POP
3. DISPLAY
4. EXIT

Enter your choice: 3

Stack elements (top to bottom):

30 20 10

MENU

1. PUSH
2. POP
3. DISPLAY
4. EXIT

Enter your choice: 2

Deleted element is 30

MENU

1. PUSH

2. POP

3. DISPLAY

4. EXIT

Enter your choice: 3

Stack elements (top to bottom):

20 10

MENU

1. PUSH

2. POP

3. DISPLAY

4. EXIT

Enter your choice: 4

Program Terminated

2. Write a C program to perform realization of dynamic Stack.

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
struct Node {  
    int data;  
    struct Node *next;  
};
```

```
struct Node *top = NULL;
```

```
/* Push operation */
```

```
void push(int value) {  
    struct Node *newNode = (struct Node*)malloc(sizeof(struct Node));
```

```
    if (newNode == NULL) {  
        printf("Stack Overflow\n");  
        return;  
    }
```

```
    newNode->data = value;  
    newNode->next = top;  
    top = newNode;  
}
```

```
/* Pop operation */
```

```
void pop() {  
    if (top == NULL) {
```

```

        printf("Stack Underflow\n");
        return;
    }

    struct Node *temp = top;
    printf("Deleted element is %d\n", temp->data);
    top = top->next;
    free(temp);
}

/* Display stack */
void display() {
    struct Node *temp = top;

    if (temp == NULL) {
        printf("Stack is empty\n");
        return;
    }

    printf("Stack elements are:\n");
    while (temp != NULL) {
        printf("%d ", temp->data);
        temp = temp->next;
    }
    printf("\n");
}

int main() {
    int choice, value;

    while (1) {
        printf("\nMENU");
        printf("\n1. PUSH");
        printf("\n2. POP");
        printf("\n3. DISPLAY");
        printf("\n4. EXIT");
        printf("\nEnter your choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                printf("Enter the value to be pushed: ");
                scanf("%d", &value);
                push(value);
                break;

            case 2:
                pop();

```

```
        break;

    case 3:
        display();
        break;

    case 4:
        printf("Program Terminated\n");
        exit(0);

    default:
        printf("Invalid choice\n");
    }
}
return 0;
}
```

Output:

```
MENU
1. PUSH
2. POP
3. DISPLAY
4. EXIT
Enter your choice: 1
Enter the value to be pushed: 10
```

```
MENU
1. PUSH
2. POP
3. DISPLAY
4. EXIT
Enter your choice: 1
Enter the value to be pushed: 20
```

```
MENU
1. PUSH
2. POP
3. DISPLAY
4. EXIT
Enter your choice: 1
Enter the value to be pushed: 30
```

```
MENU
1. PUSH
2. POP
```

3. DISPLAY
4. EXIT
Enter your choice: 3
Stack elements are:
30 20 10

MENU
1. PUSH
2. POP
3. DISPLAY
4. EXIT
Enter your choice: 2
Deleted element is 30

MENU
1. PUSH
2. POP
3. DISPLAY
4. EXIT
Enter your choice: 3
Stack elements are:
20 10

MENU
1. PUSH
2. POP
3. DISPLAY
4. EXIT
Enter your choice: 4
Program Terminated

3. Write a C program to convert an infix expression into postfix form.

```
#include <stdio.h>
#include <stdlib.h>
#include <ctype.h>

#define MAX 100

char stack[MAX];
int top = -1;

/* Function prototypes */
void push(char symbol);
char pop();
```

```

int precedence(char symbol);
void infixToPostfix(char infix[], char postfix[]);

int main() {
    char infix[MAX], postfix[MAX];

    printf("Enter an infix expression:\n");
    scanf("%s", infix); // assumes no spaces in expression

    infixToPostfix(infix, postfix);

    printf("Postfix expression:\n%s\n", postfix);

    return 0;
}

void push(char symbol) {
    if (top == MAX - 1) {
        printf("Stack overflow\n");
        exit(1);
    }
    stack[++top] = symbol;
}

char pop() {
    if (top == -1) {
        printf("Stack underflow\n");
        exit(1);
    }
    return stack[top--];
}

int precedence(char symbol) {
    switch (symbol) {
        case '^': return 3;
        case '*':
        case '/': return 2;
        case '+':
        case '-': return 1;
        default: return 0;
    }
}

void infixToPostfix(char infix[], char postfix[]) {
    int i = 0, j = 0;
    char symbol, temp;

    while (infix[i] != '\0') {

```

```

symbol = infix[i];

if (isalnum(symbol)) {      // operand
    postfix[j++] = symbol;
}
else if (symbol == '(') {
    push(symbol);
}
else if (symbol == ')') {
    while (top != -1 && stack[top] != '(') {
        postfix[j++] = pop();
    }
    pop(); // pop '('
}
else { // operator
    while (top != -1 && precedence(stack[top]) >= precedence(symbol) && stack[top] != '(') {
        postfix[j++] = pop();
    }
    push(symbol);
}
i++;
}

// pop remaining operators
while (top != -1) {
    postfix[j++] = pop();
}

postfix[j] = '\0';
}

```

Output:

Enter an infix expression:

A+B*C

Postfix expression:

ABC*+

4. Write a C program to evaluate a postfix expression.

```

#include <stdio.h>
#include <stdlib.h>
#include <ctype.h>

```

```

#define MAX 100

```

```

int stack[MAX];
int top = -1;

/* Function prototypes */
void push(int x);
int pop();
int evaluatePostfix(char postfix[]);

int main() {
    char postfix[MAX];
    int result;

    printf("Enter a postfix expression:\n");
    scanf("%s", postfix); // reads expression without spaces

    result = evaluatePostfix(postfix);

    printf("Result = %d\n", result);

    return 0;
}

void push(int x) {
    if (top == MAX - 1) {
        printf("Stack overflow\n");
        exit(1);
    }
    stack[++top] = x;
}

int pop() {
    if (top == -1) {
        printf("Stack underflow\n");
        exit(1);
    }
    return stack[top--];
}

int evaluatePostfix(char postfix[]) {
    int i = 0;
    char symbol;
    int op1, op2, value;

    while (postfix[i] != '\0') {
        symbol = postfix[i];

        if (isdigit(symbol)) { // operand

```

```

        value = symbol - '0';    // convert char to int
        push(value);
    } else {                      // operator
        op2 = pop();
        op1 = pop();

        switch (symbol) {
            case '+':
                push(op1 + op2);
                break;
            case '-':
                push(op1 - op2);
                break;
            case '*':
                push(op1 * op2);
                break;
            case '/':
                push(op1 / op2);
                break;
            case '%':
                push(op1 % op2);
                break;
            default:
                printf("Invalid operator %c\n", symbol);
                exit(1);
        }
    }
    i++;
}

return pop(); // final result
}

```

Output:

Enter a postfix expression:

23*54*+9-

Result = 17

5. Write a C program to validate an infix expression.

```
#include <stdio.h>
```

```
#include <ctype.h>
```

```
#define MAX 100
```

```

char stack[MAX];
int top = -1;

/* Stack operations for parentheses */
void push(char ch) {
    if (top == MAX - 1) {
        printf("Stack overflow\n");
        return;
    }
    stack[++top] = ch;
}

char pop() {
    if (top == -1) {
        return '\0';
    }
    return stack[top--];
}

int isEmpty() {
    return (top == -1);
}

/* Function to validate infix expression */
int isValidInfix(char expr[]) {
    int i = 0;
    char ch;
    int prevType = 0;
    /*
    prevType:
    0 -> start
    1 -> operand
    2 -> operator
    3 -> '('
    4 -> ')'
    */

    while ((ch = expr[i]) != '\0') {
        if (isalnum(ch)) { /* operand */
            if (prevType == 1 || prevType == 4) {
                /* two operands in a row or operand after ')' */
                return 0;
            }
            prevType = 1;
        }
        else if (ch == '+' || ch == '-' || ch == '*' || ch == '/' || ch == '%') { /* operator */
            if (prevType == 0 || prevType == 2 || prevType == 3) {

```

```

        /* cannot start with operator or have two operators or operator after '(' */
        return 0;
    }
    prevType = 2;
}
else if (ch == '(') {
    if (prevType == 1 || prevType == 4) {
        /* operand or ')' directly before '(' is not allowed */
        return 0;
    }
    push(ch);
    prevType = 3;
}
else if (ch == ')') {
    if (prevType == 0 || prevType == 2 || prevType == 3) {
        /* cannot have ')' at start, after operator, or after '(' */
        return 0;
    }
    if (isEmpty()) {
        /* no matching '(' */
        return 0;
    }
    pop();
    prevType = 4;
}
else {
    /* invalid character */
    return 0;
}
i++;
}

/* at end: stack must be empty & last token not operator or '(' or start */
if (!isEmpty())
    return 0;
if (prevType == 0 || prevType == 2 || prevType == 3)
    return 0;

return 1; /* valid */
}

int main() {
    char expr[MAX];

    printf("Enter an infix expression:\n");
    scanf("%s", expr); /* assumes no spaces in expression */

    if (isValidInfix(expr))

```

```

        printf("Valid infix expression\n");
    else
        printf("Invalid infix expression\n");

    return 0;
}

```

Output 1:

```

Enter an infix expression:
(A+B)*C-D
Valid infix expression

```

6. Write a C program to implement Student stack.

```

#include <stdio.h>
#include <stdlib.h>

struct Student {
    int regNo;
    char name[50];
    int marks;
    struct Student *next;
};

struct Student *top = NULL;

/* Function to push student */
void push() {
    struct Student *newNode = (struct Student *)malloc(sizeof(struct Student));

    if (newNode == NULL) {
        printf("Stack Overflow\n");
        return;
    }

    printf("Enter Register Number: ");
    scanf("%d", &newNode->regNo);

    printf("Enter Name: ");
    scanf("%s", newNode->name);

    printf("Enter Marks: ");
    scanf("%d", &newNode->marks);

    newNode->next = top;
}

```

```

    top = newNode;

    printf("Student pushed into stack\n");
}

/* Function to pop student */
void pop() {
    if (top == NULL) {
        printf("Stack Underflow\n");
        return;
    }

    struct Student *temp = top;
    printf("Popped Student:\n");
    printf("RegNo: %d, Name: %s, Marks: %d\n",
        temp->regNo, temp->name, temp->marks);

    top = top->next;
    free(temp);
}

/* Display stack */
void display() {
    if (top == NULL) {
        printf("Stack is empty\n");
        return;
    }

    struct Student *temp = top;
    printf("\nStudent Stack (Top to Bottom):\n");

    while (temp != NULL) {
        printf("RegNo: %d | Name: %s | Marks: %d\n",
            temp->regNo, temp->name, temp->marks);
        temp = temp->next;
    }
}

/* Main function */
int main() {
    int choice;

    while (1) {
        printf("\nSTUDENT STACK MENU");
        printf("\n1. PUSH (Add Student)");
        printf("\n2. POP (Remove Student)");
        printf("\n3. DISPLAY");
        printf("\n4. EXIT");
    }
}

```

```

printf("\nEnter your choice: ");
scanf("%d", &choice);

switch (choice) {
    case 1:
        push();
        break;
    case 2:
        pop();
        break;
    case 3:
        display();
        break;
    case 4:
        printf("Program Terminated\n");
        exit(0);
    default:
        printf("Invalid choice\n");
}
}
return 0;
}

```

Output:

STUDENT STACK MENU

1. PUSH (Add Student)
2. POP (Remove Student)
3. DISPLAY
4. EXIT

Enter your choice: 1

Enter Register Number: 101

Enter Name: Ravi

Enter Marks: 85

Student pushed into stack

STUDENT STACK MENU

1. PUSH (Add Student)
2. POP (Remove Student)
3. DISPLAY
4. EXIT

Enter your choice: 1

Enter Register Number: 102

Enter Name: Anita

Enter Marks: 90

Student pushed into stack

STUDENT STACK MENU

1. PUSH (Add Student)
2. POP (Remove Student)
3. DISPLAY
4. EXIT

Enter your choice: 3

Student Stack (Top to Bottom):

RegNo: 102 | Name: Anita | Marks: 90

RegNo: 101 | Name: Ravi | Marks: 85

STUDENT STACK MENU

1. PUSH (Add Student)
2. POP (Remove Student)
3. DISPLAY
4. EXIT

Enter your choice: 2

Popped Student:

RegNo: 102, Name: Anita, Marks: 90

STUDENT STACK MENU

1. PUSH (Add Student)
2. POP (Remove Student)
3. DISPLAY
4. EXIT

Enter your choice: 4

Program Terminated

7. Write a C program to implement Employee stack. Employee Details - Employee No,

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
struct Employee {  
    int empNo;  
    char name[50];  
    char designation[50];  
    float salary;  
    struct Employee *next;  
};
```

```
struct Employee *top = NULL;
```

```

/* Push Employee */
void push() {
    struct Employee *newNode = (struct Employee *)malloc(sizeof(struct Employee));

    if (newNode == NULL) {
        printf("Stack Overflow\n");
        return;
    }

    printf("Enter Employee Number: ");
    scanf("%d", &newNode->empNo);

    printf("Enter Employee Name: ");
    scanf("%s", newNode->name);

    printf("Enter Designation: ");
    scanf("%s", newNode->designation);

    printf("Enter Salary: ");
    scanf("%f", &newNode->salary);

    newNode->next = top;
    top = newNode;

    printf("Employee pushed into stack\n");
}

/* Pop Employee */
void pop() {
    if (top == NULL) {
        printf("Stack Underflow\n");
        return;
    }

    struct Employee *temp = top;

    printf("Removed Employee:\n");
    printf("EmpNo: %d | Name: %s | Designation: %s | Salary: %.2f\n",
        temp->empNo, temp->name, temp->designation, temp->salary);

    top = top->next;
    free(temp);
}

/* Display Stack */
void display() {
    if (top == NULL) {
        printf("Stack is empty\n");
    }
}

```

```

        return;
    }

    struct Employee *temp = top;
    printf("\nEmployee Stack (Top to Bottom):\n");

    while (temp != NULL) {
        printf("EmpNo: %d | Name: %s | Designation: %s | Salary: %.2f\n",
            temp->empNo, temp->name, temp->designation, temp->salary);
        temp = temp->next;
    }
}

/* Main */
int main() {
    int choice;

    while (1) {
        printf("\nEMPLOYEE STACK MENU");
        printf("\n1. PUSH (Add Employee)");
        printf("\n2. POP (Remove Employee)");
        printf("\n3. DISPLAY");
        printf("\n4. EXIT");
        printf("\nEnter your choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                push();
                break;
            case 2:
                pop();
                break;
            case 3:
                display();
                break;
            case 4:
                printf("Program Terminated\n");
                exit(0);
            default:
                printf("Invalid choice\n");
        }
    }
    return 0;
}

```

Output:

EMPLOYEE STACK MENU

1. PUSH (Add Employee)
2. POP (Remove Employee)
3. DISPLAY
4. EXIT

Enter your choice: 1

Enter Employee Number: 201

Enter Employee Name: Kumar

Enter Designation: Manager

Enter Salary: 45000

Employee pushed into stack

EMPLOYEE STACK MENU

1. PUSH (Add Employee)
2. POP (Remove Employee)
3. DISPLAY
4. EXIT

Enter your choice: 1

Enter Employee Number: 202

Enter Employee Name: Priya

Enter Designation: Developer

Enter Salary: 38000

Employee pushed into stack

EMPLOYEE STACK MENU

1. PUSH (Add Employee)
2. POP (Remove Employee)
3. DISPLAY
4. EXIT

Enter your choice: 3

Employee Stack (Top to Bottom):

EmpNo: 202 | Name: Priya | Designation: Developer | Salary: 38000.00

EmpNo: 201 | Name: Kumar | Designation: Manager | Salary: 45000.00

EMPLOYEE STACK MENU

1. PUSH (Add Employee)
2. POP (Remove Employee)
3. DISPLAY
4. EXIT

Enter your choice: 2

Removed Employee:

EmpNo: 202 | Name: Priya | Designation: Developer | Salary: 38000.00

EMPLOYEE STACK MENU

1. PUSH (Add Employee)

2. POP (Remove Employee)

3. DISPLAY

4. EXIT

Enter your choice: 4

Program Terminated

8. Write a C program to perform realization of integer static Queue.

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#define QSIZE 5
```

```
struct queue {  
    int front;  
    int rear;  
    int item[QSIZE];  
} q;
```

```
void insert(int x);
```

```
int delete();
```

```
void display();
```

```
int isEmpty();
```

```
int isFull();
```

```
int main() {  
    int choice, x, del;
```

```
    q.front = 0;
```

```
    q.rear = -1;
```

```
    while (1) {  
        printf("\nMENU");  
        printf("\n1. INSERT");  
        printf("\n2. DELETE");  
        printf("\n3. DISPLAY");  
        printf("\n4. EXIT");  
        printf("\nEnter your choice: ");  
        scanf("%d", &choice);
```

```
        switch (choice) {  
            case 1:  
                printf("Enter the element to be inserted: ");  
                scanf("%d", &x);  
                insert(x);  
                break;
```

```

        case 2:
            del = delete();
            if (del != -1)
                printf("Deleted element is %d\n", del);
            break;

        case 3:
            display();
            break;

        case 4:
            printf("Program Terminated\n");
            exit(0);

        default:
            printf("Invalid choice\n");
    }
}
return 0;
}

```

```

void insert(int x) {
    if (isFull()) {
        printf("Queue overflow\n");
    } else {
        q.item[++q.rear] = x;
    }
}
}

```

```

int delete() {
    if (isEmpty()) {
        printf("Queue underflow\n");
        return -1;
    } else {
        return q.item[q.front++];
    }
}
}

```

```

void display() {
    int i;
    if (isEmpty()) {
        printf("Queue is empty\n");
        return;
    }
}

```

```

printf("Queue elements are:\n");
for (i = q.front; i <= q.rear; i++) {

```

```

        printf("%d ", q.item[i]);
    }
    printf("\n");
}

int isEmpty() {
    return (q.front > q.rear);
}

int isFull() {
    return (q.rear == QSIZE - 1);
}

```

Output:

```

MENU
1. INSERT
2. DELETE
3. DISPLAY
4. EXIT
Enter your choice: 1
Enter the element to be inserted: 10

```

```

MENU
1. INSERT
2. DELETE
3. DISPLAY
4. EXIT
Enter your choice: 1
Enter the element to be inserted: 20

```

```

MENU
1. INSERT
2. DELETE
3. DISPLAY
4. EXIT
Enter your choice: 1
Enter the element to be inserted: 30

```

```

MENU
1. INSERT
2. DELETE
3. DISPLAY
4. EXIT
Enter your choice: 3
Queue elements are:

```

10 20 30

MENU

1. INSERT
2. DELETE
3. DISPLAY
4. EXIT

Enter your choice: 2

Deleted element is 10

MENU

1. INSERT
2. DELETE
3. DISPLAY
4. EXIT

Enter your choice: 3

Queue elements are:

20 30

MENU

1. INSERT
2. DELETE
3. DISPLAY
4. EXIT

Enter your choice: 4

Program Terminated

9. Write a C program to perform realization of dynamic Queue.

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
struct Node {  
    int data;  
    struct Node *next;  
};
```

```
struct Node *front = NULL;  
struct Node *rear = NULL;
```

```
/* Insert (Enqueue) */
```

```
void insert(int value) {  
    struct Node *newNode = (struct Node *)malloc(sizeof(struct Node));
```

```
    if (newNode == NULL) {  
        printf("Queue Overflow\n");
```

```

        return;
    }

    newNode->data = value;
    newNode->next = NULL;

    if (rear == NULL) {
        front = rear = newNode;
    } else {
        rear->next = newNode;
        rear = newNode;
    }
}

/* Delete (Dequeue) */
void delete() {
    if (front == NULL) {
        printf("Queue Underflow\n");
        return;
    }

    struct Node *temp = front;
    printf("Deleted element is %d\n", temp->data);

    front = front->next;
    if (front == NULL)
        rear = NULL;

    free(temp);
}

/* Display */
void display() {
    if (front == NULL) {
        printf("Queue is empty\n");
        return;
    }

    struct Node *temp = front;
    printf("Queue elements are:\n");

    while (temp != NULL) {
        printf("%d ", temp->data);
        temp = temp->next;
    }
    printf("\n");
}

```

```
int main() {
    int choice, value;

    while (1) {
        printf("\nMENU");
        printf("\n1. INSERT");
        printf("\n2. DELETE");
        printf("\n3. DISPLAY");
        printf("\n4. EXIT");
        printf("\nEnter your choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                printf("Enter the element to be inserted: ");
                scanf("%d", &value);
                insert(value);
                break;

            case 2:
                delete();
                break;

            case 3:
                display();
                break;

            case 4:
                printf("Program Terminated\n");
                exit(0);

            default:
                printf("Invalid choice\n");
        }
    }
    return 0;
}
```

Output:

```
MENU
1. INSERT
2. DELETE
3. DISPLAY
4. EXIT
Enter your choice: 1
```

Enter the element to be inserted: 10

MENU

1. INSERT
2. DELETE
3. DISPLAY
4. EXIT

Enter your choice: 1

Enter the element to be inserted: 20

MENU

1. INSERT
2. DELETE
3. DISPLAY
4. EXIT

Enter your choice: 1

Enter the element to be inserted: 30

MENU

1. INSERT
2. DELETE
3. DISPLAY
4. EXIT

Enter your choice: 3

Queue elements are:

10 20 30

MENU

1. INSERT
2. DELETE
3. DISPLAY
4. EXIT

Enter your choice: 2

Deleted element is 10

MENU

1. INSERT
2. DELETE
3. DISPLAY
4. EXIT

Enter your choice: 3

Queue elements are:

20 30

MENU

1. INSERT
2. DELETE
3. DISPLAY

4. EXIT

Enter your choice: 4

Program Terminated

10. Write a C program to implement Student Queue.

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
struct Student {  
    int regNo;  
    char name[50];  
    int marks;  
    struct Student *next;  
};
```

```
struct Student *front = NULL;
```

```
struct Student *rear = NULL;
```

```
/* Insert Student (Enqueue) */
```

```
void insert() {
```

```
    struct Student *newNode = (struct Student *)malloc(sizeof(struct Student));
```

```
    if (newNode == NULL) {  
        printf("Queue Overflow\n");  
        return;  
    }
```

```
    printf("Enter Register Number: ");  
    scanf("%d", &newNode->regNo);
```

```
    printf("Enter Name: ");  
    scanf("%s", newNode->name);
```

```
    printf("Enter Marks: ");  
    scanf("%d", &newNode->marks);
```

```
    newNode->next = NULL;
```

```
    if (rear == NULL) {  
        front = rear = newNode;  
    } else {  
        rear->next = newNode;  
        rear = newNode;  
    }
```

```

    printf("Student inserted into queue\n");
}

/* Delete Student (Dequeue) */
void delete() {
    if (front == NULL) {
        printf("Queue Underflow\n");
        return;
    }

    struct Student *temp = front;

    printf("Removed Student:\n");
    printf("RegNo: %d | Name: %s | Marks: %d\n",
        temp->regNo, temp->name, temp->marks);

    front = front->next;

    if (front == NULL)
        rear = NULL;

    free(temp);
}

/* Display Queue */
void display() {
    if (front == NULL) {
        printf("Queue is empty\n");
        return;
    }

    struct Student *temp = front;
    printf("\nStudent Queue (Front to Rear):\n");

    while (temp != NULL) {
        printf("RegNo: %d | Name: %s | Marks: %d\n",
            temp->regNo, temp->name, temp->marks);
        temp = temp->next;
    }
}

/* Main Function */
int main() {
    int choice;

    while (1) {
        printf("\nSTUDENT QUEUE MENU");
    }
}

```

```

printf("\n1. INSERT (Enqueue)");
printf("\n2. DELETE (Dequeue)");
printf("\n3. DISPLAY");
printf("\n4. EXIT");
printf("\nEnter your choice: ");
scanf("%d", &choice);

switch (choice) {
    case 1:
        insert();
        break;
    case 2:
        delete();
        break;
    case 3:
        display();
        break;
    case 4:
        printf("Program Terminated\n");
        exit(0);
    default:
        printf("Invalid choice\n");
}
}
return 0;
}

```

Output:

STUDENT QUEUE MENU

1. INSERT (Enqueue)
2. DELETE (Dequeue)
3. DISPLAY
4. EXIT

Enter your choice: 1

Enter Register Number: 101

Enter Name: Ravi

Enter Marks: 85

Student inserted into queue

STUDENT QUEUE MENU

1. INSERT (Enqueue)
2. DELETE (Dequeue)
3. DISPLAY
4. EXIT

Enter your choice: 1

Enter Register Number: 102
Enter Name: Anita
Enter Marks: 92
Student inserted into queue

STUDENT QUEUE MENU

1. INSERT (Enqueue)
2. DELETE (Dequeue)
3. DISPLAY
4. EXIT

Enter your choice: 3

Student Queue (Front to Rear):

RegNo: 101 | Name: Ravi | Marks: 85

RegNo: 102 | Name: Anita | Marks: 92

STUDENT QUEUE MENU

1. INSERT (Enqueue)
2. DELETE (Dequeue)
3. DISPLAY
4. EXIT

Enter your choice: 2

Removed Student:

RegNo: 101 | Name: Ravi | Marks: 85

STUDENT QUEUE MENU

1. INSERT (Enqueue)
2. DELETE (Dequeue)
3. DISPLAY
4. EXIT

Enter your choice: 4

Program Terminated

11. Write a C program to implement Employee Queue.

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
struct Employee {  
    int empNo;  
    char name[50];  
    char designation[50];  
    float salary;  
    struct Employee *next;  
};
```

```
struct Employee *front = NULL;
```

```

struct Employee *rear = NULL;

/* Insert Employee (Enqueue) */
void insert() {
    struct Employee *newNode = (struct Employee *)malloc(sizeof(struct Employee));

    if (newNode == NULL) {
        printf("Queue Overflow\n");
        return;
    }

    printf("Enter Employee Number: ");
    scanf("%d", &newNode->empNo);

    printf("Enter Employee Name: ");
    scanf("%s", newNode->name);

    printf("Enter Designation: ");
    scanf("%s", newNode->designation);

    printf("Enter Salary: ");
    scanf("%f", &newNode->salary);

    newNode->next = NULL;

    if (rear == NULL) {
        front = rear = newNode;
    } else {
        rear->next = newNode;
        rear = newNode;
    }

    printf("Employee inserted into queue\n");
}

/* Delete Employee (Dequeue) */
void delete() {
    if (front == NULL) {
        printf("Queue Underflow\n");
        return;
    }

    struct Employee *temp = front;

    printf("Removed Employee:\n");
    printf("EmpNo: %d | Name: %s | Designation: %s | Salary: %.2f\n",
        temp->empNo, temp->name, temp->designation, temp->salary);
}

```

```

front = front->next;

if (front == NULL)
    rear = NULL;

free(temp);
}

/* Display Queue */
void display() {
    if (front == NULL) {
        printf("Queue is empty\n");
        return;
    }

    struct Employee *temp = front;
    printf("\nEmployee Queue (Front to Rear):\n");

    while (temp != NULL) {
        printf("EmpNo: %d | Name: %s | Designation: %s | Salary: %.2f\n",
            temp->empNo, temp->name, temp->designation, temp->salary);
        temp = temp->next;
    }
}

/* Main Function */
int main() {
    int choice;

    while (1) {
        printf("\nEMPLOYEE QUEUE MENU");
        printf("\n1. INSERT (Enqueue)");
        printf("\n2. DELETE (Dequeue)");
        printf("\n3. DISPLAY");
        printf("\n4. EXIT");
        printf("\nEnter your choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                insert();
                break;
            case 2:
                delete();
                break;
            case 3:
                display();
                break;

```

```
        case 4:
            printf("Program Terminated\n");
            exit(0);
        default:
            printf("Invalid choice\n");
    }
}
return 0;
}
```

Output:

EMPLOYEE QUEUE MENU

1. INSERT (Enqueue)
2. DELETE (Dequeue)
3. DISPLAY
4. EXIT

Enter your choice: 1

Enter Employee Number: 301

Enter Employee Name: Kumar

Enter Designation: Manager

Enter Salary: 50000

Employee inserted into queue

EMPLOYEE QUEUE MENU

1. INSERT (Enqueue)
2. DELETE (Dequeue)
3. DISPLAY
4. EXIT

Enter your choice: 1

Enter Employee Number: 302

Enter Employee Name: Priya

Enter Designation: Developer

Enter Salary: 42000

Employee inserted into queue

EMPLOYEE QUEUE MENU

1. INSERT (Enqueue)
2. DELETE (Dequeue)
3. DISPLAY
4. EXIT

Enter your choice: 3

Employee Queue (Front to Rear):

EmpNo: 301 | Name: Kumar | Designation: Manager | Salary: 50000.00

EmpNo: 302 | Name: Priya | Designation: Developer | Salary: 42000.00

EMPLOYEE QUEUE MENU

1. INSERT (Enqueue)
2. DELETE (Dequeue)
3. DISPLAY
4. EXIT

Enter your choice: 2

Removed Employee:

EmpNo: 301 | Name: Kumar | Designation: Manager | Salary: 50000.00

EMPLOYEE QUEUE MENU

1. INSERT (Enqueue)
2. DELETE (Dequeue)
3. DISPLAY
4. EXIT

Enter your choice: 4

Program Terminated

Assignment-4

1. Write a recursive C program to find factorial of a given number.

```
#include <stdio.h>

/* Recursive function to find factorial */
int factorial(int n) {
    if (n == 0 || n == 1)
        return 1;
    else
        return n * factorial(n - 1);
}

int main() {
    int n, result;

    printf("Enter a number: ");
    scanf("%d", &n);

    if (n < 0) {
        printf("Factorial not defined for negative numbers\n");
    } else {
        result = factorial(n);
        printf("Factorial of %d = %d\n", n, result);
    }

    return 0;
}
```

Output:

```
Enter a number: 5
Factorial of 5 = 120
```

2. Write a recursive C program to find Fibonacci value of a given number.

```
#include <stdio.h>

/* Recursive function to find Fibonacci */
int fibonacci(int n) {
    if (n == 0)
        return 0;
    else if (n == 1)
        return 1;
    else
        return fibonacci(n - 1) + fibonacci(n - 2);
}
```

```

int main() {
    int n, result;

    printf("Enter the position: ");
    scanf("%d", &n);

    if (n < 0) {
        printf("Fibonacci not defined for negative numbers\n");
    } else {
        result = fibonacci(n);
        printf("Fibonacci value at position %d = %d\n", n, result);
    }

    return 0;
}

```

Output:

Enter the position: 7
 Fibonacci value at position 7 = 13

3. Write a recursive C program to find multiplication of two numbers.

```

#include <stdio.h>

/* Recursive function to find product */
int product(int a, int b) {
    if (b == 0)
        return 0;
    else
        return a + product(a, b - 1);
}

int main() {
    int a, b, result;

    printf("Enter two numbers: ");
    scanf("%d %d", &a, &b);

    result = product(a, b);

    printf("Product of %d and %d = %d\n", a, b, result);

    return 0;
}

```

```
}
```

Output:

Enter two numbers: 6 4
Product of 6 and 4 = 24

4. Write a recursive C program to find GCD of two numbers.

```
#include <stdio.h>
```

```
/* Recursive function to find GCD using Euclidean algorithm */
```

```
int gcd(int a, int b) {  
    if (b == 0)  
        return a;  
    else  
        return gcd(b, a % b);  
}
```

```
int main() {  
    int a, b, result;  
  
    printf("Enter two numbers: ");  
    scanf("%d %d", &a, &b);  
  
    result = gcd(a, b);  
  
    printf("GCD of %d and %d = %d\n", a, b, result);  
  
    return 0;  
}
```

Output:

Enter two numbers: 48 18
GCD of 48 and 18 = 6

5. Write a recursive C program to solve Towers of Hanoi problem.

```
#include <stdio.h>
```

```
/* Recursive function for Towers of Hanoi */
```

```
void toh(int n, char source, char destination, char auxiliary) {
```

```
if (n == 1) {
    printf("Move disk 1 from %c to %c\n", source, destination);
    return;
}

toh(n - 1, source, auxiliary, destination);
printf("Move disk %d from %c to %c\n", n, source, destination);
toh(n - 1, auxiliary, destination, source);
}

int main() {
    int n;

    printf("Enter number of disks: ");
    scanf("%d", &n);

    printf("Steps to solve Towers of Hanoi:\n");
    toh(n, 'A', 'C', 'B');

    return 0;
}
```

Output:

```
Enter number of disks: 3
Steps to solve Towers of Hanoi:
Move disk 1 from A to C
Move disk 2 from A to B
Move disk 1 from C to B
Move disk 3 from A to C
Move disk 1 from B to A
Move disk 2 from B to C
Move disk 1 from A to C
```

Assignment-5

1. Write a C program to sort an array of n elements using Bubble Sort Technique.

```
#include <stdio.h>

int main() {
    int n, i, j, temp;
    int a[100];

    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter %d elements:\n", n);
    for (i = 0; i < n; i++) {
        scanf("%d", &a[i]);
    }

    /* Bubble Sort Logic */
    for (i = 0; i < n - 1; i++) {
        for (j = 0; j < n - i - 1; j++) {
            if (a[j] > a[j + 1]) {
                temp = a[j];
                a[j] = a[j + 1];
                a[j + 1] = temp;
            }
        }
    }

    printf("Sorted array:\n");
    for (i = 0; i < n; i++) {
        printf("%d ", a[i]);
    }

    return 0;
}
```

Output:

```
Enter number of elements: 5
Enter 5 elements:
45 12 78 3 25
Sorted array:
3 12 25 45 78
```

2. Write a C program to sort an array of n elements using Selection Sort Technique.

```
#include <stdio.h>
```

```

int main() {
    int n, i, j, min, temp;
    int a[100];

    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter %d elements:\n", n);
    for (i = 0; i < n; i++) {
        scanf("%d", &a[i]);
    }

    /* Selection Sort Logic */
    for (i = 0; i < n - 1; i++) {
        min = i;
        for (j = i + 1; j < n; j++) {
            if (a[j] < a[min]) {
                min = j;
            }
        }

        if (min != i) {
            temp = a[i];
            a[i] = a[min];
            a[min] = temp;
        }
    }

    printf("Sorted array:\n");
    for (i = 0; i < n; i++) {
        printf("%d ", a[i]);
    }

    return 0;
}

```

Output:

```

Enter number of elements: 5
Enter 5 elements:
45 12 78 3 25
Sorted array:
3 12 25 45 78

```

3. Write a C program to sort an array of n elements using Quick Sort Technique.

```
#include <stdio.h>
```

```
/* Function to swap two elements */
```

```

void swap(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}

/* Partition function */
int partition(int a[], int low, int high) {
    int pivot = a[high]; /* choosing last element as pivot */
    int i = low - 1;
    int j;

    for (j = low; j < high; j++) {
        if (a[j] < pivot) {
            i++;
            swap(&a[i], &a[j]);
        }
    }
    swap(&a[i + 1], &a[high]);
    return i + 1;
}

/* Quick Sort function */
void quickSort(int a[], int low, int high) {
    if (low < high) {
        int pi = partition(a, low, high);

        quickSort(a, low, pi - 1);
        quickSort(a, pi + 1, high);
    }
}

int main() {
    int n, i;
    int a[100];

    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter %d elements:\n", n);
    for (i = 0; i < n; i++) {
        scanf("%d", &a[i]);
    }

    quickSort(a, 0, n - 1);

    printf("Sorted array:\n");
    for (i = 0; i < n; i++) {

```

```

        printf("%d ", a[i]);
    }

    return 0;
}

```

Output:

```

Enter number of elements: 6
Enter 6 elements:
45 12 78 3 25 10
Sorted array:
3 10 12 25 45 78

```

4. Write a C program to sort an array of n elements using Simple Insertion Sort Technique.

```
#include <stdio.h>
```

```

int main() {
    int n, i, j, key;
    int a[100];

    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter %d elements:\n", n);
    for (i = 0; i < n; i++) {
        scanf("%d", &a[i]);
    }

    /* Insertion Sort Logic */
    for (i = 1; i < n; i++) {
        key = a[i];
        j = i - 1;

        while (j >= 0 && a[j] > key) {
            a[j + 1] = a[j];
            j--;
        }
        a[j + 1] = key;
    }

    printf("Sorted array:\n");
    for (i = 0; i < n; i++) {
        printf("%d ", a[i]);
    }

    return 0;
}

```

```
}
```

Output:

Enter number of elements: 5

Enter 5 elements:

45 12 78 3 25

Sorted array:

3 12 25 45 78

5. Write a C program to sort an array of n elements using Shell Sort Technique.

```
#include <stdio.h>
```

```
int main() {
    int n, i, j, temp;
    int a[100];

    scanf("%d", &n);
    for (i = 0; i < n; i++) scanf("%d", &a[i]);

    int m, gaps[50];
    scanf("%d", &m);
    for (i = 0; i < m; i++) scanf("%d", &gaps[i]);

    for (int g = 0; g < m; g++) {
        int gap = gaps[g];
        if (gap >= n) continue;

        for (i = gap; i < n; i++) {
            if (a[i - gap] > a[i]) {
                temp = a[i];
                a[i] = a[i - gap];
                a[i - gap] = temp;
            }
        }
    }

    for (i = gap; i < n; i++) {
        temp = a[i];
        j = i;
        while (j >= gap && a[j - gap] > temp) {
            a[j] = a[j - gap];
            j -= gap;
        }
        a[j] = temp;
    }
}

for (i = 0; i < n; i++) printf("%d ", a[i]);
```

```
    return 0;
}
```

Output:

```
Enter number of elements: 8
Enter 8 elements:
23 12 1 8 34 54 2 3
Enter number of gaps in the gap-sequence: 3
Enter 3 gap values:
4 2 1
Sorted array:
1 2 3 8 12 23 34 54
```

6. Write a C program to sort an array of n elements using Merge Sort Technique.

```
#include <stdio.h>

/* Function to merge two subarrays */
void merge(int a[], int low, int mid, int high) {
    int i = low;
    int j = mid + 1;
    int k = low;
    int temp[100];

    while (i <= mid && j <= high) {
        if (a[i] <= a[j])
            temp[k++] = a[i++];
        else
            temp[k++] = a[j++];
    }

    while (i <= mid)
        temp[k++] = a[i++];

    while (j <= high)
        temp[k++] = a[j++];

    for (i = low; i <= high; i++)
        a[i] = temp[i];
}

/* Merge Sort function */
void mergeSort(int a[], int low, int high) {
    int mid;
```

```

    if (low < high) {
        mid = (low + high) / 2;

        mergeSort(a, low, mid);
        mergeSort(a, mid + 1, high);
        merge(a, low, mid, high);
    }
}

int main() {
    int n, i;
    int a[100];

    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter %d elements:\n", n);
    for (i = 0; i < n; i++) {
        scanf("%d", &a[i]);
    }

    mergeSort(a, 0, n - 1);

    printf("Sorted array:\n");
    for (i = 0; i < n; i++) {
        printf("%d ", a[i]);
    }

    return 0;
}

```

Output:

```

Enter number of elements: 6
Enter 6 elements:
45 12 78 3 25 10
Sorted array:
3 10 12 25 45 78

```

Assignment-6

1.write a c program to search an element from an array of elements using sequential search technique

```
#include <stdio.h>

int main() {
    int a[100], n, i, key, pos = -1;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter %d elements:\n", n);
    for (i = 0; i < n; i++) {
        scanf("%d", &a[i]);
    }

    printf("Enter element to search: ");
    scanf("%d", &key);

    /* Sequential Search */
    for (i = 0; i < n; i++) {
        if (a[i] == key) {
            pos = i;
            break;
        }
    }

    if (pos == -1)
        printf("Element %d not found\n", key);
    else
        printf("Element %d found at position %d\n", key, pos);

    return 0;
}
```

Output:

```
Enter number of elements: 5
Enter 5 elements:
10 25 32 7 18
Enter element to search: 32
Element 32 found at position 2
```

2.write a c program to search an element from an array of n elements using binary search technique

```
#include <stdio.h>
```

```
int main() {
```

```

int a[100], n, key, low, high, mid, i, pos = -1;

printf("Enter number of elements: ");
scanf("%d", &n);

printf("Enter %d elements (sorted):\n", n);
for (i = 0; i < n; i++) {
    scanf("%d", &a[i]);
}

printf("Enter element to search: ");
scanf("%d", &key);

low = 0;
high = n - 1;

/* Binary Search Logic */
while (low <= high) {
    mid = (low + high) / 2;

    if (a[mid] == key) {
        pos = mid;
        break;
    }
    else if (key < a[mid]) {
        high = mid - 1;
    }
    else {
        low = mid + 1;
    }
}

if (pos == -1)
    printf("Element %d not found\n", key);
else
    printf("Element %d found at position %d\n", key, pos);

return 0;
}

```

Output:

```

Enter number of elements: 6
Enter 6 elements (sorted):
5 12 18 27 45 60
Enter element to search: 27
Element 27 found at position 3

```

3.write a c program to search an element from an array of n elements using interpolation search technique

```
#include <stdio.h>

/* Function for Interpolation Search */
int interpolationSearch(int a[], int n, int key) {
    int low = 0;
    int high = n - 1;
    int pos;

    while (low <= high && key >= a[low] && key <= a[high]) {
        if (low == high) {
            if (a[low] == key)
                return low;
            return -1;
        }

        /* Estimate the position */
        pos = low + ((key - a[low]) * (high - low)) / (a[high] - a[low]);

        if (a[pos] == key)
            return pos;

        if (a[pos] < key)
            low = pos + 1;
        else
            high = pos - 1;
    }

    return -1;
}

int main() {
    int a[100], n, i, key, result;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter %d elements (sorted):\n", n);
    for (i = 0; i < n; i++) {
        scanf("%d", &a[i]);
    }

    printf("Enter element to search: ");
    scanf("%d", &key);
```

```
result = interpolationSearch(a, n, key);

if (result == -1)
    printf("Element %d not found\n", key);
else
    printf("Element %d found at position %d\n", key, result);

return 0;
}
```

Output:

```
Enter number of elements: 6
Enter 6 elements (sorted):
10 20 30 40 50 60
Enter element to search: 40
Element 40 found at position 3
```